# COMPUTER NETWORKS VIVA & PRESENTATION MASTER GUIDE

**Student:** Sarah Singh • **Roll No:** 35615602724 • **Project:** Custom DNS Resolver from Scratch

## SECTION 0: THE 30-SECOND 'AURA' OPENING SCRIPT

*Jab presentation start ho, bina hichkichaye confident hoke ye bolo. Teacher pehli line me hi impress ho jayega:*

**Exact English Script:**
“Good morning Sir/Ma'am, my project is a **Custom DNS Resolver built from scratch in Python**. Most developers resolve domains using black-box libraries like *dnspython* or system calls. In this project, I deliberately avoided all third-party libraries and implemented the **RFC 1035 wire-protocol directly over raw UDP datagram sockets (Port 53)**.
It constructs binary packet headers, handles 14-bit DNS name compression pointers, implements an **Iterative Resolution loop** that manually queries the 13 global Root servers down to Authoritative nameservers, and features an in-memory **TTL caching engine** that drops latency from 380 milliseconds to **0.0 milliseconds**.”

## SECTION 1: BILKUL DESI BASICS (No-BS Networking Concepts)

| | |
|---|---|
| **1. DNS Kya Hai?** | Internet ki **Truecaller / Phonebook**. Insaan naam yaad rakhte hain (google.com), machines sirf number (IP address: 142.250.x.x) samajhti hain. DNS naam ko number me convert karta hai. |
| **2. Packet Kya Hota Hai?** | Internet par data ek saath nahi behata, chhote-chhote lifaafe (envelopes) me jaata hai jise **Packet / Datagram** kehte hain. Isme do hisse hote hain: **Header** (lifaafe pe likha address aur flags) aur **Body / Data** (jo actual message hai). |
| **3. Port 53 Kya Hai?** | Computer ek building hai jisme 65,535 khidkiyan (ports) hain. DNS ka permanent adda **Port 53** hai. Saare DNS servers port 53 par khade hoke request ka intezaar karte hain. |
| **4. UDP vs TCP?** | **TCP** phone call jaisa hai (pehle hello-hello 3-way handshake hoti hai, phir baat hoti hai - safe but slow).<br>**UDP** postcard jaisa hai (bina handshake seedha fek diya - super fast!). DNS ko fast hona zaroori hai, isliye DNS **UDP** use karta hai. |

## SECTION 2: HAMARA PROJECT ASLIYAT MEIN KYA KARTA HAI? (Step-by-Step)

**Jab hum likhte hain: python3 cli.py github.com A --trace, andar ye 5 steps hote hain:**
**Step 1 (Diary Check):** Code pehle apni RAM diary (Cache) me dekhta hai ki kya github.com pehle se hai? Agar haan, toh 0.0ms me answer wahi se de deta hai.
**Step 2 (Packet Crafting):** Agar nahi mila, toh 'github.com' ko RFC 1035 format ke 28 hex bytes me pack karta hai (12-byte header + label bytes \x06github\x03com\x00).
**Step 3 (UDP Socket):** Python socket banata hai aur Internet ke 13 Root Servers me se ek (e.g. 198.41.0.4) par packet UDP port 53 pe fekta hai.
**Step 4 (Iterative Loop - Referral Chasing):**
- **Root Server** bolta hai: *'Mujhe github nahi pata, par .com TLD server 192.52.178.30 se poocho.'*
- **TLD Server** bolta hai: *'Main .com hoon, github ke authoritative nameserver 205.251.194.8 se poocho.'*
- **Authoritative Server** bolta hai: *'Haan yeh lo github ka exact IP: **135.235.149.144**!'*

**Step 5 (Parsing & Caching):** Code raw binary bytes ko decode karta hai, IP nikaalta hai, diary (cache) me TTL expiry ke saath likhta hai, aur screen pe print karta hai.

## SECTION 3: FILE ANATOMY (Kon si file kya karti hai)

| File Name | Role in Project (Desi Samajh) | Key Code Concept |
|---|---|---|
| `dns_packet.py` | **The Translator:** Domain string ko hex bytes me pack karta hai aur server se aane wale binary bytes ko decode karta hai. | struct.pack('!HHHHHH'), 0xC0 compression pointer parser. |
| `dns_resolver.py` | **The Engine / Postman:** Asli networking yahan hoti hai. UDP socket banata hai, port 53 pe bhejta hai, Root ➔ TLD ➔ Auth loop chalata hai. | socket.socket(AF_INET, SOCK_DGRAM), Iterative while loop. |
| `dns_cache.py` | **The Memory Diary:** Resolved IPs ko TTL timestamp ke saath store karta hai taaki repeat query 0ms me ho sake. | Thread-safe dictionary, TTL expiration logic. |
| `cli.py` | **Terminal Interface:** Terminal me dig-style colored trace aur comparison output dikhata hai. | Argument parsing (--trace, --compare). |
| `web_ui.py` | **Visual Dashboard:** Flask web server jo localhost:8080 par aesthetic card-based UI dikhata hai. | Flask REST endpoints (/api/resolve, /api/compare). |
| `dns_server.py` | **Local Server Daemon:** PC ko port 5353 par mini DNS server bana deta hai (dig command isse baat kar sakti hai). | UDP socket bind to 127.0.0.1:5353. |

## SECTION 4: TOP 10 VIVA QUESTIONS + AURA ANSWERS (The Exam Lifesaver)

*Har question ke do answer hain: ek normal concept, aur ek 'Aura Answer' (agar blank ho jao toh bina soche bas wahi bol dena!).*

**[IMPORTANT / MUST DO]**

**Q1. Tumne Python ki bani-banayi library (dnspython / scapy) kyu use nahi ki?**

**Normal Explanation:** Bani-banayi library sirf high-level call hoti hai jo andar ke binary protocol ko chhupa deti hai. Humne RFC 1035 wire format khud byte-by-byte struct module se banaya hai taaki Application Layer aur Transport Layer socket programming ki internal working prove ho sake.

■ **Aura Answer (Bina soche ye boldo):** "Sir, dnspython use karne se to 2 line ka code banta. Main objective RFC 1035 binary wire-format aur raw UDP socket programming ko first principles se implement karna tha."

**[CRITICAL CONCEPT]**

**Q2. DNS UDP kyu use karta hai, TCP kyu nahi?**

**Normal Explanation:** 1. Speed: UDP connectionless hai, 3-way handshake ka overhead nahi hota. 2. Lightweight: DNS request < 512 bytes hoti hai toh single datagram me aati hai. 3. Scalability: Root servers par lakho connections ka TCP state rakhna impossible hai. (Note: TCP tab use hota hai jab response 512B se bada ho TC flag ke sath ya Zone Transfer AXFR ho).

■ **Aura Answer (Bina soche ye boldo):** "Sir, UDP avoids the 3-way handshake latency overhead and standard DNS queries fit in a single 512-byte datagram. TCP is only reserved for zone transfers and truncated responses."

**[MUST DO]**

**Q3. Iterative aur Recursive resolution me kya farak hai?**

**Normal Explanation: Recursive (RD=1):** Client ek server (8.8.8.8) ko bolta hai 'Mujhe final answer laake do, poora kaam tum karo'.

**Iterative (RD=0):** Client khud ek-ek karke Root ➔ TLD ➔ Authoritative ke paas jata hai referrals follow karte hue.

■ **Aura Answer (Bina soche ye boldo):** "Sir, Recursive me delegation server par hoti hai with RD=1 bit, jabki Iterative me client khud Root, TLD aur Authoritative hierarchy ko referrals ke through walk karta hai."

**[HIGH CHANCE]**

**Q4. TTL (Time-To-Live) kya hota hai aur tumhara cache kaise kaam karta hai?**

**Normal Explanation:** TTL ek 32-bit integer hai jo batata hai record kitne seconds valid rahega. Humara resolver resolved IP ko local dictionary me store karta hai. Agar time < TTL hai toh network pe jaye bina instant 0.0ms me return karta hai.

■ **Aura Answer (Bina soche ye boldo):** "Sir, TTL prevents stale DNS entries and reduces root-server load. Our in-memory cache drops lookup latency from 380ms to sub-millisecond 0.0ms."

**[EASY MARKS]**

**Q5. DNS Packet Header ka size aur structure kya hota hai?**

**Normal Explanation:** Exact 12 bytes fixed size: Transaction ID (16-bit), Flags (16-bit: QR, Opcode, AA, TC, RD, RA, RCODE), aur 4 section counters (QDCOUNT, ANCOUNT, NSCOUNT, ARCOUNT - har ek 16-bit).

■ **Aura Answer (Bina soche ye boldo):** "Sir, exactly 12 bytes in big-endian network byte order containing a 16-bit ID, 16-bit control flags, and four 16-bit count fields."

**[TRICKY QUESTION]**

**Q6. Domain Name ko binary me kaise encode karte hain (QNAME)?**

**Normal Explanation:** Dots se split karke har label ke aage uski length ka byte lagate hain aur aakhri me zero byte (0x00) lagate hain. Jaise 'www.google.com' banta hai: \x03www\x06google\x03com\x00.

■ **Aura Answer (Bina soche ye boldo):** "Sir, length-prefixed label format: each label is preceded by its 1-byte length and terminated with a null byte 0x00."

**[HARD / PROFESSOR FAVORITE]**

**Q7. DNS Compression Pointer (0xC0) kya hota hai?**

**Normal Explanation:** Domain names baar-baar repeat hote hain. Space bachane ke liye RFC 1035 rule banata hai: Agar length byte ke pehle 2 bits '11' (hex 0xC0) hain, toh baaki ke 14 bits ek pointer hote hain jo packet ke purane offset ki taraf point karte hain jahan wo naam pehle likha tha.

■ **Aura Answer (Bina soche ye boldo):** "Sir, if the first two bits are 11 (0xC0), the remaining 14 bits act as an offset pointer to a previously decoded name in the packet, preventing redundant string repetitions."

**[CONCEPTUAL]**

**Q8. NXDOMAIN aur SERVFAIL me kya farak hai?**

**Normal Explanation: NXDOMAIN (RCODE 3):** Server chal raha hai, par domain exist hi nahi karta.

**SERVFAIL (RCODE 2):** Server par koi internal issue aa gaya (e.g. timeout ya misconfiguration) aur wo resolve nahi kar paya.

■ **Aura Answer (Bina soche ye boldo):** “Sir, NXDOMAIN means the domain does not exist in the authoritative zone, while SERVFAIL indicates the nameserver encountered an internal failure or upstream timeout.”

**[EASY / GENERAL]**

**Q9. Tumhara code kaun-kaun se Record Types support karta hai?**

**Normal Explanation:** A (IPv4), AAAA (IPv6), MX (Mail Exchange server), NS (Nameserver delegation), CNAME (Canonical Name alias), aur TXT (Domain verification strings).

■ **Aura Answer (Bina soche ye boldo):** “Sir, we support A, AAAA, MX, NS, CNAME, and TXT records, each parsed according to its specific RDATA schema.”

**[EDGE CASE]**

**Q10. Agar UDP packet network me drop ho jaye toh tumhara code kya karega?**

**Normal Explanation:** Humne socket par 5.0 seconds ka timeout lagaya hai (`sock.settimeout(5.0)`). Agar packet drop ho jaye toh `socket.timeout` exception catch hota hai, error log hoti hai, aur code crash nahi hota.

■ **Aura Answer (Bina soche ye boldo):** “Sir, socket timeout exception handling ensures that if an upstream datagram is dropped, the resolver fails gracefully without blocking the thread.”

## SECTION 5: LIVE DEMO PLAYBOOK (Screen par kya run karke dikhana hai)

*Professor ke bolte hi yeh 3 commands run karke dikhana:*

**Demo 1: The Iterative Trace Flex**
**Command:** python3 cli.py github.com A --trace
Screen pe Root Server (192.33.4.12) -> .com TLD -> AWS Nameserver teen steps aayenge.
**Kya bolna hai:** *'Sir dekhiye, yahan koi shortcut nahi liya. Step 1 me Root server ne referral diya, Step 2 me TLD server ne referral diya, aur Step 3 me Authoritative server ne final IP diya.'*

**Demo 2: The Cache Speedup Flex**
**Command:** python3 cli.py github.com A
Same command bina trace ke chalao. Output me likha aayega: **Time: 0.0ms | Source: Cache Hit ✓**.
**Kya bolna hai:** *'Sir pehli baar me 380ms lage the, second time humare TTL-cache ne bina network call kiye 0.0ms me memory se return kar diya.'*

**Demo 3: Latency Benchmark vs System DNS**
**Command:** python3 cli.py google.com --compare
Side-by-side comparison table aayega showing Iterative vs Recursive vs Host OS DNS.
**Kya bolna hai:** *'Yahan hum apne resolver ki performance host operating system ke getaddrinfo resolver se compare kar rahe hain.'*

**Demo 4: Web UI Dashboard**
**Command:** python3 web_ui.py (Open http://localhost:8080)
Sleek modern card-based dashboard khulega.
**Kya bolna hai:** *'Sir, demonstration ke liye humne ek responsive web dashboard bhi banaya hai jisme hop timeline aur live cache inspection supported hai.'*

## SECTION 6: EMERGENCY BAILOUT SCRIPT (Jab Examiner kuch ajeeb pooch le)

Agar examiner koi high-level advanced topic pooch le (jaise DNSSEC, Anycast BGP routing, EDNS0, DoH):
**Step 1: Ghabrana mat. Bilkul chup mat rehna.**
**Step 2: Yeh line bolo:**
*“Sir, that is related to advanced DNS extensions like DNSSEC cryptographic verification and EDNS0 buffer scaling. In our core scope, we strictly prioritized mastering RFC 1035 fundamental socket mechanics and binary serialization. However, as mentioned in Chapter 9 of our report under Future Scope, DNSSEC and DNS-over-HTTPS are our direct next milestones.”*
➔ **Result:** Examiner sochenga ki bache ko na sirf basics aate hain, balki aage ka roadmap bhi pata hai!